

中山大學

SUN YAT-SEN UNIVERSITY

实验 0x03 实验报告

内核线程与缺页异常

姓　　名：　何亮

学　　号：　24325005

教学班号：　202520804（陈鹏飞老师班级）

专　　业：　计算机科学与技术

院　　系：　计算机学院

2025 学年第二学期

1. Checkpoint 1

该部分我们需要初始化进程管理器和实现内存调度。

首先是初始化进程管理器，我们需要先设置内核进程的信息，这里设置了如下三个信息：页表大小，内核堆的大小，以及内核进程对应的 PID

```
let mut kdata = ProcessData::new();
    kdata.set_env("PAGE_SIZE", &PAGE_SIZE.to_string());
    kdata.set_env("KERNEL_HEAP_SIZE",
&crate::memory::allocator::HEAP_SIZE.to_string());
    kdata.set_env("KERNEL_PID", "1");
```

然后我们根据实验文档调用 `Process::new` 函数来创建内核进程并传入进程管理器，来初始化进程管理器并加入内核进程作为第一个进程

```
let kproc = Process::new(
        "kernel".into(),
        None,
        Some(proc_vm),
        Some(kdata),
    );
```

随后，我们要重写时钟中断，使用其中断处理函数来进行进程的调度。首先在 `memory/gdt.rs` 中为其创建独立的栈，然后再在 `clock.rs` 中 `.set_stack_index(gdt::CLOCK_IST_INDEX)` 为时钟中断设置专门的栈。这个中断处理程序只需要调用 `proc::switch()` 函数委托调度任务即可，然后使用 `as_handler!` 宏进行打包。

```
pub extern "C" fn clock(mut context: ProcessContext) {
    super::ack();
    crate::proc::switch(&mut context);
}

as_handler!(clock);
```

我们的 `proc::switch()` 函数只需要进一步把保存上下文和调度的任务委托给 `save_current()` 和 `switch_next()` 函数即可。

对于 `save_current()` 函数，我们需要将当前进程的时间片 +1，以及保存当前进程的上下文，同时将当前进程设置为 `Ready` 就绪状态，并放入就绪队列，如下

```
// FIXED: update current process's tick count
proc_inner.tick();
// FIXED: save current process's context
proc_inner.save(context);

if !was_dead {
  proc_inner.pause();

  drop(proc_inner); // 这里提前释放锁，避免 switch_next 自旋等待锁的时间过长

  self.push_ready(pid);
}
```

对于 `switch_next()` 函数，需要在就绪队列中取出最前一个确实处于 `Ready` 就绪状态的进程，同时更新该进程的 PID 并返回，代码如下

```
loop {
    // FIXED: fetch the next process from ready queue
    let next_pid = {
        let mut ready_queue = self.ready_queue.lock();
        ready_queue.pop_front().unwrap_or(KERNEL_PID)
    };
    // FIXED: check if the next process is ready,
    //        continue to fetch if not ready
    let Some(next_proc) = self.get_proc(&next_pid) else {
        continue;
    };
    let mut next_inner = next_proc.write();
    if !next_inner.is_ready() && next_pid != KERNEL_PID {
        continue;
    }
    // FIXED: restore next process's context
    next_inner.restore(context);
    drop(next_inner);
    // FIXED: update processor's current pid
    processor::set_pid(next_pid);
    // FIXED: return next process's pid
    return next_pid;
}
```

然后我们运行 `make run`，使用 `ps` 命令可以观察到内核进程确实在运行当中，时钟也可以正常计数。

```
(base) linkzelda@LAPTOP-QFAEHS2I:~/OS-lab$ make run
[INFO] ysos_kernel::memory::gdt - Clock IST        : 0xffffff00008243c0-0xffffff00008253c0
[INFO] ysos_kernel::memory::gdt - Kernel IST Size  :  12.000 KiB
[INFO] ysos_kernel::memory::gdt - GDT Initialized.
[INFO] ysos_kernel::interrupt - Interrupts Initialized.
[INFO] ysos_kernel::memory - Physical Memory    :  95.625 MiB
[INFO] ysos_kernel::memory - Free Usable Memory :  44.941 MiB
[INFO] ysos_kernel::memory - Frame Allocator initialized.
[INFO] ysos_kernel::proc - Process Manager Initialized.
[INFO] ysos_kernel - Interrupts Enabled.
[INFO] ysos_kernel - YatSenOS initialized.
[>] ps
  PID | PPID | Process Name |  Ticks  | Status
 #  1 | #  0 | kernel       |    6441 | Running
Queue  : []
CPUs   : [0: 1]
[>] exit
[INFO] ysos_kernel - YatSenOS shutting down.
(base) linkzelda@LAPTOP-QFAEHS2I:~/OS-lab$
```

2. Checkpoint 2

这部分我们要实现的是获取进程信息和创建内核进程。

对于其环境变量的返回，只需要获取当前进程的读锁，然后使用内部提供的 `.env(key)` 函数即可

```
get_process_manager().current().read().env(key)
```

对于 `wait` 函数，我通过两步委托，最终在 `ProcessManager` 内部实现 `exit_code` 的获取，返回值是 `Option` 类型，用于判断进程是否退出，在 `wait` 内部使用循环等待进程退出得到返回值。`wait` 函数如下，其余部分参见 `proc/mod.rs` 和 `proc/manager.rs`

```
fn wait(pid: ProcessId) {
    loop {
        // FIXED: try to get the status of the process
        let exit_code = crate::proc::get_process_exit_code(pid);
        // HINT: it's better to use the exit code
        if exit_code.is_none() {
            x86_64::instructions::hlt();
        } else {
            break;
        }
    }
}
```

对于内核线程的创建，只需要将 `spawn_kernel_thread` 函数内部需要完成的任务向下委托到对应的内部函数即可

```
// alloc stack for the new process base on pid
let stack_top = proc.alloc_init_stack();
// FIXED: set the stack frame
proc.write().init_stack_frame(entry, stack_top);
// FIXED: add to process map
self.add_proc(pid, proc);
// FIXED: push to ready queue
self.push_ready(pid);
// FIXED: return new process pid
pid
```

对于栈帧的初始化，我参考实验文档中给出的内存布局进行了初始化，如下

```
pub fn alloc_init_stack(&mut self, pid: ProcessId, mapper: MapperRef) -> VirtAddr
{
      let alloc = &mut *get_frame_alloc_for_sure();

      let pid_offset = (pid.0 as u64 - 1) * STACK_MAX_SIZE;
      let stack_bot = STACK_INIT_BOT - pid_offset;
      let stack_top = STACK_INIT_TOP - pid_offset;

      self.range = map_range(stack_bot, STACK_DEF_PAGE, mapper, alloc)
          .expect("Failed to map stack");

      VirtAddr::new(stack_top)
  }
```

到此，我们运行 `make run`，执行若干个 `test` 用 `ps` 观察，可以观察到内核线程被创建同时被管理器进行调度（进程编号按顺序闪烁，符合预期的切换顺序）

```
(base) linkzelda@LAPTOP-QFAEHS2I:~/OS-lab$ make run
[INFO] ysos_kernel - Interrupts Enabled.
[INFO] ysos_kernel - YatSenOS initialized.
[>] test
0      => Tick!st
1      => Tick!st
2      => Tick!su
  PID | PPID | Process Name |  Ticks  | Status
 #  2 | #  1 | #0_test      |   20892 | Ready
 #  3 | #  1 | #1_test      |   14778 | Ready
 #  1 | #  0 | kernel       |   33572 | Running
 #  4 | #  1 | #2_test      |    6401 | Ready
Queue  : [2, 3, 4]
CPUs   : [0: 1]
2      => Tick!xit
[INFO] ysos_kernel - YatSenOS shutting down.
(base) linkzelda@LAPTOP-QFAEHS2I:~/OS-lab$
```

我们注意到，如果单纯使用 `ps` 命令查看，每次都是 kernel 在 `Running` 其他进程都是 `Ready`，这是因为每次处理我们这个命令的都是内核进程，我们可以通过在 `utils/func.rs` 里面的 `test` 使用 `crate::proc::print_process_list()` 来观察其他进程是否 `Running` 过。我们在 `main.rs` 初始化内核之后马上创建五个内核进程，顺便观察各进程时间是否大致平均

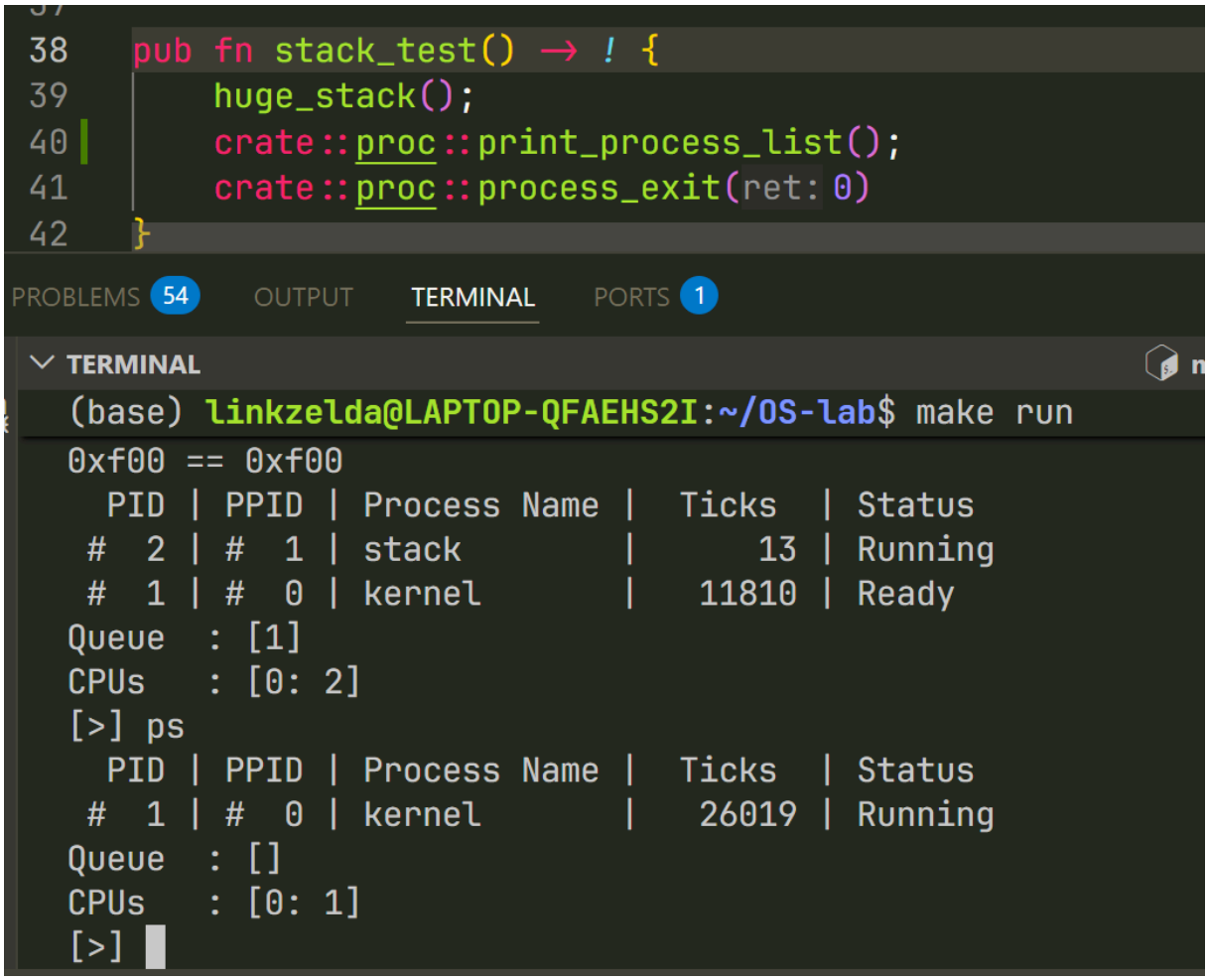
```
(base) linkzelda@LAPTOP-QFAEHS2I:~/OS-lab$ make run
[>]   PID | PPID | Process Name |  Ticks   | Status
 #   2 | #   1 | #0_test      |      12 | Ready
 #   6 | #   1 | #4_test      |       0 | Running
 #   5 | #   1 | #3_test      |       4 | Ready
 #   3 | #   1 | #1_test      |       8 | Ready
 #   1 | #   0 | kernel       |      33 | Ready
 #   4 | #   1 | #2_test      |       7 | Ready
Queue  : [1, 2, 3, 4, 5]
CPUs   : [0: 6]
4      => Tick!ps
  PID | PPID | Process Name |  Ticks   | Status
 #   2 | #   1 | #0_test      |    1676 | Ready
 #   6 | #   1 | #4_test      |    1665 | Ready
 #   5 | #   1 | #3_test      |    1668 | Ready
 #   3 | #   1 | #1_test      |    1672 | Ready
 #   1 | #   0 | kernel       |    1697 | Running
 #   4 | #   1 | #2_test      |    1671 | Ready
Queue  : [2, 3, 4, 5, 6]
CPUs   : [0: 1]
0      => Tick!
```

如图可知，就绪队列正常，各进程的 status 显示正常，占用 CPU 的时间也大致平均。

我们可以在创建进程栈空间的时候使用 `log::info!` 输出栈信息，如下，可以看到栈顶和栈底地址均与预期一致（PID 2 的初始栈顶地址为 `0x00003ffeffffffff8`）

```
[INFO] ysos_kernel::proc - Process Manager Initialized.
[INFO] ysos_kernel - Interrupts Enabled.
[INFO] ysos_kernel - YatSenOS initialized.
[>] test
[INFO] ysos_kernel::proc::vm::stack - Stack of Process 2 : 0x00003ffefffff000 ~
 0x00003ffeffffffff8
0      => Tick!st
[INFO] ysos_kernel::proc::vm::stack - Stack of Process 3 : 0x00003ffdfffff000 ~
 0x00003ffdffffffff8
1      => Tick!
```

我们在栈测试的内部输出进程状态，可以看到在栈测试销毁进程后，进程不再在就绪队列中出现，没有继续执行



3. Checkpoint 3

这部分我们要实现缺页异常的处理以及进程的退出。

根据实验文档，我们只需要实现栈空间的扩容，如下，

```
let fault_page = Page::containing_address(addr);
let current_start = self.range.start;

if fault_page >= current_start { // 判断目前缺页的地址是否合法（是否发生越界等）
    return Ok(());
}

let page_to_alloc = current_start - fault_page; // 计算出还要分配的页数
let start_addr = fault_page.start_address().as_u64();

map_range(start_addr, page_to_alloc, mapper, alloc)?; // 建立页表映射

self.range = Page::range(fault_page, self.range.end); // 更新已经映射了的栈的范围
self.usage += page_to_alloc; // 更新站的的物理页面数
```

然后我们需要实现进程的退出，我们只需要实现 `ProcessInner` 内部的处理，此部分的代码如下

```
// FIXED: set exit code
self.exit_code = Some(ret);

// FIXED: set status to dead
self.status = ProgramStatus::Dead;

// FIXED: take and drop unused resources
self.proc_data.take();
self.proc_vm.take();
```

即需要设置返回值，把该进程的状态设为 `Dead` 并且删除进程的数据即可。我们在之前的 `save_current` 函数里面判断了进程状态是否为 `Dead`，如非 `Dead` 才能加入就绪队列，保证杀死的进程不会再被调度。

此时我们 `make run` 一下，尝试运行 `stack` 测试缺页中断处理程序，可以看到，确实扩展了栈空间，使得可以存储这些数组中的值并输出

```
(base) linkzelda@LAPTOP-QFAEHS2I:~/OS-lab$ make run
[INFO] ysos_kernel - Interrupts Enabled.
[INFO] ysos_kernel - YatSenOS initialized.
[>] stack
Huge stack testing...
0x000 == 0x000
0x100 == 0x100
0x200 == 0x200
0x300 == 0x300
0x400 == 0x400
0x500 == 0x500
0x600 == 0x600
0x700 == 0x700
0x800 == 0x800
0x900 == 0x900
0xa00 == 0xa00
0xb00 == 0xb00
0xc00 == 0xc00
0xd00 == 0xd00
0xe00 == 0xe00
0xf00 == 0xf00
  PID | PPID | Process Name |  Ticks  | Status
 #  2 | #  1 | stack        |      18 | Running
 #  1 | #  0 | kernel       |   18998 | Ready
Queue  : [1]
CPUs   : [0: 2]
[>]
```

4. 思考题

4.1. 为什么在初始化进程管理器时需要将它置为正在运行的状态? 能否通过将它置为就绪状态并放入就绪队列来实现? 这样的实现可能会遇到什么问题?

在初始化进程管理器的时候，内核处于 `Running` 状态，而作为 `init` 传入的进程对应的就是内核进程，因此需要将它置为正在运行的状态。

不可以将它置为 `Ready` 就绪状态并放入就绪队列来实现，这样实现可能会造成在时钟中断时，中断处理程序保存了当前进程的上下文并放入就绪队列，此时原本的内核进程尚未弹出，这样会造成有两个相同的内核进程放在就绪队列中，可能导致调度器出错。

4.2. 在 `src/proc/process.rs` 中，有两次实现 `Deref` 和一次实现 `DerefMut` 的代码，它们分别是为了什么? 使用这种方式提供了什么便利?

`Deref` 和 `DerefMut` 是用来隐式解引用的, 可以便于该类型的变量实例更方便地访问内部成员, 前者是不可变引用, 后者是可变引用。在 `process.rs` 里面为 `Process` 和 `ProcessInner` 类型设置了 `Deref`，同时为后者设置了 `DerefMut`。这样可以减少重复的代码，同时给出了封装得更加安全的访问接口。

如对于一个 `Process` 类型变量 `proc` 而言，我们要调用其内部 `inner` 的成员函数，可以直接使用 `proc.func()` 来调用，编译器会首先寻找 `Process` 是否存在该成员函数，若无则隐式调用 `proc.deref().func()`；而 `DerefMut` 返回的是 `self.proc_data.as_mut().expect("Process data empty. The process may be killed.")`，在返回可变引用的同时也进行了特殊处理, 可以大大简化代码, 我们不需要关系具体内部的功能结构，直接调用最外层的成员即可被编译器处理，提升了类型的抽象，便于代码编写。

4.3. 中断的处理过程默认是不切换栈的, 即在中断发生前的栈上继续处理中断过程, 为什么在处理缺页异常和时钟中断时需要切换栈? 如果不为它们切换栈会分别带来哪些问题? 请假设具体的场景、或通过实际尝试进行回答。

首先，对于缺页异常，由于缺页的原因有很多，一种情况是进程的栈溢出，此时如果我们按照默认不切换栈，那么就会把当前上下文保存在当前缺页的栈中，导致出现 double fault 和 triple fault，进而导致内核直接崩溃重启，因此我们需要为其设置独立的栈。

其次，对于时间中断，由于其中断的频率较高，在程序运行的各个阶段均有可能发生中断。假设我们在运行某个进程的时候，此时栈的已经快满了，那么如果此时来了一个时钟中断，则会由于要运行中断处理程序而造成栈溢出, 如果不为其设置一个独立的栈, 则可能会带来很多概率性发生的错误，大大增加调试人员的工作量。

4.4. 结合本实验中的线程调度，比较以下两种策略：

- 仅在时钟中断时触发调度；
- 在时钟中断 + 阻塞唤醒点共同触发调度。

讨论它们对响应延迟、公平性和实现复杂度的影响。

从响应延迟来看，前者的延迟较高。比如如果某个进程（设为 1）由于一个外部事情导致忙等待，前者的策略只能等到时间片结束，才能调度到下一个进程；而后者可以在忙等待的时候主动让出 CPU 给其他进程，提高了 CPU 的利用率，降低了响应的延迟。

从公平性来看，前者更为公平，其保证了每个进程占用 CPU 资源的时间公平。查阅资料，现代操作系统会在后者策略的基础上配合“优先级+时间片”策略，提升主动让出 CPU 的进程的优先级，实现“动态公平”。

从实现的复杂度来看，前者的复杂度更低，因为我们只需要维护一个队列，然后每次时钟中断时切换上下文进行进程调度即可；而后者需要加入一个 `Blocked` 阻塞状态，且需要维护多一个等待队列，同时会多很多特殊情况需要考虑和处理。

4.5. 若调度器改为“优先级 + 时间片”混合策略，你认为当前数据结构最先需要改动的部分是什么?

首先我们要修改 `ready_queue`，由于我们加入了优先级策略，因此需要改成一个支持按优先级存储的数据结构（如优先队列，平衡树等）；其次还要修改 PCB 的数据结构，在数据类型中加入优先级权重，便于按照优先级进行排序。

5. 遇到的问题

在实现了时钟中断之后，运行 `make run` 发现会卡住，没办法输入字符，后来经过排查，发现是 `get_line()` 函数中的 `let mut line = String::with_capacity(128);` 卡死，经过大模型的帮助发现是因为 `allocator.rs` 采用了自旋锁来保护堆，当主线程需要 `get_line` 的时候获得了锁，此时时钟中断发生，有其他的进程采用自旋的方式等待堆的锁，导致死锁。

最后重写了一下 `allocator.rs`，确保在每次 `alloc` 和 `dealloc` 时，强制关闭系统的硬件时钟中断，让当前线程“不受任何打扰地”切完内存再放开锁。